

Low-Code Programming Models

An Annotated Bibliography

Tianyiming Jing
University of Michigan, CSE 590

April 20, 2026

1 Introduction

Software plays a role in nearly every human activity, yet only a small fraction of people know how to create it. Low-code programming models aim to close this gap. Instead of writing conventional textual code, users express programs through visual interfaces, demonstrations, examples, or natural language. Although the term “low-code” is recent, the underlying research spans more than six decades across programming languages, human-computer interaction, and artificial intelligence.

Among these approaches, *programming by demonstration* (PBD) and *programming by example* (PBE) are especially promising. Both avoid requiring users to learn any notation, but they differ in what the user must provide. In PBD, the user performs the desired task directly (clicking, copying, editing), and the system watches those *actions* and learns to replicate them. For example, a user who wants to number items in a list performs the first two iterations by hand. The system detects the pattern and automates the rest. In PBE, the user does not perform any actions at all. Instead, they provide *input-output pairs* that show what the program should produce. For example, a user shows the system that “John Smith” should become “Smith, J.” The system then applies the same rule to every name in the column. The key distinction is that PBD captures the *process* of doing a task, while PBE captures only the *desired result*. This bibliography traces the evolution of PBD and PBE from early heuristic systems, to formal frameworks, to industrial tools used by millions. It also briefly covers visual programming and natural-language programming to place PBD and PBE in the broader low-code landscape.

2 Summary of Results

Non-textual programming dates back to Sutherland’s *Sketchpad* [7] in 1963. Sketchpad let users draw shapes on a screen and define constraints among them, establishing the paradigm of *visual programming*. Later systems such as Scratch brought block-based visual programming to millions of children. Visual programming is simpler than writing code, but it still demands that users learn a new notation, whether blocks, wires, or diagrams. This learning cost is what PBD and PBE set out to eliminate.

In *programming by demonstration* (PBD), the user simply performs a task, and the system watches and learns to automate it. One of the earliest PBD systems was Cypher’s *EAGER* [2], which ran silently in the background while users worked in HyperCard. When EAGER detected a repeating pattern of actions, it highlighted the predicted next step and offered to complete the remaining iterations automatically. This showed that a system could learn from observation alone, without any explicit programming. However, EAGER’s pattern detection used simple heuristics and could not handle conditional logic or nested loops.

Lau et al. [5] addressed this limitation by giving PBD a formal theoretical foundation. They proposed *version space algebra*, a framework that compactly represents every program consistent with a user’s demonstrations. The key idea is to decompose the program space into independent subspaces connected by algebraic operators. Each new demonstration then refines the space without recomputing from scratch. Their SMARTedit system learned text-editing macros from one to three demonstrations, showing that PBD could be treated as a principled learning problem rather than a collection of heuristics.

Building on this work, Gulwani [3] introduced *FlashFill*, a system that showed program synthesis could work at an industrial scale. FlashFill shifts from PBD to *programming by example* (PBE): instead of watching the user perform a task step by step, the system only asks for input-output pairs. A user who wants to reformat phone numbers, for instance, types “(617) 253-0000” $\rightarrow$ “617-253-0000”, and FlashFill synthesizes a program that transforms every remaining row the same way. No actions are recorded; only the desired result matters. The system defines a domain-specific language of string expressions and uses version-space intersection to find all matching programs. Integrated into Microsoft Excel 2013, it reached millions of end users.

After FlashFill’s success, a natural next step was to ask: could the same approach work across different domains? Polozov and Gulwani [6] answered with *FlashMeta*, a framework for building PBE systems. Instead of writing a custom synthesis algorithm for each new domain, a developer defines a DSL and provides *witness functions* that describe how to propagate examples through each operator. FlashMeta then derives the synthesis algorithm automatically. Re-implementations of FlashFill and other PBE tools using FlashMeta matched the originals’ performance with two to eight times fewer lines of code. Microsoft has since used it to ship multiple PBE features across different products.

Meanwhile, large language models opened a third path: programming through natural language. Chen et al. [1] introduced *Codex*, a GPT model fine-tuned on public code from GitHub that generates Python functions from English docstrings. On the HumanEval benchmark, Codex solved 28.8% of problems on the first attempt and 70.2% with repeated sampling. Codex went on to power GitHub Copilot, bringing natural-language code generation into mainstream use.

Hirzel [4] brings these different lines of work together in a unified taxonomy. He classifies low-code systems by notation (visual, demonstrative, natural language) and identifies shared design challenges such as intent disambiguation and liveness. Any low-code system must grapple with these challenges, regardless of its notation.

3 Open Research Questions

Several important questions remain open, particularly for PBD and PBE.

First, *correctness and trust* remain unsolved. When examples are ambiguous, FlashFill can synthesize a program that fits the examples but does not match the user’s intent. The user has no easy way to check whether the result is correct without inspecting many outputs by hand. FlashMeta carries this problem into every domain it supports. How to help users verify synthesized programs efficiently is still an open question.

Second, most PBD and PBE research has focused on *end-user productivity domains* like spreadsheets and text editing. Extending these techniques to *expert domains* such as formal verification, scientific computing, and hardware design is largely unexplored. Tools like PGSynth, which uses PBD for hardware memory-consistency verification, show that the approach can reach beyond its traditional scope. But designing PBD/PBE systems for expert domains is not straightforward. These domains require richer specification languages and stronger correctness guarantees than current tools provide.

Third, *combining modalities* remains largely unexplored. A user might sketch a visual diagram, supply a few examples, and then clarify with a natural-language instruction. Designing interfaces that support such mixed inputs smoothly is still an open challenge.

Finally, *scalability* is a concern. Current tools handle isolated, small-scale tasks well, but applying them to large, evolving codebases is much harder. Whether PBD and PBE can grow beyond single-task automation to support larger software systems remains to be seen.

4 Updated Implementation Proposal

For the implementation component, I am contributing to PGSynth, a low-code tool being developed in Prof. Yatin Manerkar’s research group at the University of Michigan. PGSynth synthesizes μ Spec axioms for microarchitectural memory consistency verification. Users interact through a visual graph interface, drawing edges between pipeline stages to specify ordering constraints. The backend then uses the CVC5 SMT solver to synthesize the corresponding formal axioms. In this way, PGSynth applies programming by demonstration to an expert domain.

My work focuses on implementing and maintaining the backend API that connects the visual frontend to the synthesis engine. Specifically, I am building the REST endpoints that the frontend uses to retrieve microarchitectural happens-before (UHB) graphs, run quantifier disambiguation on user-drawn constraints, and edit or delete synthesized axioms. These endpoints take the user’s visual demonstrations (graphs with nodes for pipeline stages and edges for ordering relations) and convert them into inputs for the CVC5 solver. The solver’s output is then returned to the frontend as updated graphs. More endpoints will be added as the tool’s functionality grows.

A key technical challenge has been understanding how the existing codebase translates axiom specifications into CVC5 solver queries and interprets the results back as graphs. The legacy code for this translation is complex. Working through it required building a detailed understanding of both the μ Spec axiom format and the CVC5 API. This difficulty connects to the open research questions discussed above: most low-code and PBD systems target end-user productivity domains like spreadsheets, while PGSynth applies the same ideas to formal hardware verification, where users need deep expertise in formal methods.

References

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.

Large language models had shown strong results on natural-language tasks, but their ability to generate correct code was still untested. Chen et al. test this by fine-tuning a GPT model on code. The authors introduce Codex, a GPT-based model fine-tuned on publicly available code from GitHub. Given a natural-language docstring, Codex generates a corresponding Python function body. The model comes in several sizes (up to 12 billion parameters). Performance can be further improved through repeated sampling: generating multiple candidates and filtering them against test cases. The main evaluation uses HumanEval, a new benchmark of 164 hand-written Python problems with unit tests. The best Codex model solves 28.8% of problems on the first attempt (pass@1) and 70.2% when allowed 100 samples (pass@100). The paper also reports results on the APPS benchmark and documents scaling laws: larger models and more training data

yield consistent gains. Codex struggles with long, multi-step reasoning and sometimes produces plausible but incorrect code. It offers no formal correctness guarantees, and the paper discusses broader risks including insecure code generation. Future work would need to address reliability, verification, and alignment with user intent.

- [2] A. Cypher, “EAGER: Programming repetitive tasks by example,” in *Proc. ACM SIGCHI Conference on Human Factors in Computing Systems (CHI)*, 1991, pp. 33–39.

Macro recorders existed at the time, but they required users to explicitly start and stop recording. They also could not generalize beyond the exact recorded actions. Cypher asks whether a system can automate repetitive tasks without the user even knowing that automation is happening. The solution is EAGER, a programming-by-demonstration (PBD) agent that runs in the background while the user works in HyperCard. When EAGER detects that the user is repeating a pattern of actions, it highlights the next predicted action and offers to complete the remaining iterations automatically. The system uses heuristics to generalize from the observed actions. For example, it can recognize when the user is iterating over a list rather than clicking on a fixed set of locations. Cypher evaluates EAGER through a user study in which seven subjects were each given three repetitive HyperCard tasks to perform, without being told anything about EAGER. When subjects performed the tasks correctly, EAGER detected the repeating pattern in 17 out of 21 task instances. EAGER is limited to a small set of heuristics for detecting patterns, and it cannot handle tasks with complex conditionals or nested loops. It also depends on the specific HyperCard environment. Future work would need to replace the heuristics with a more general learning mechanism and support a wider range of applications.

- [3] S. Gulwani, “Automating string processing in spreadsheets using input-output examples,” in *Proc. 38th ACM SIGPLAN-SIGACT Symp. Principles of Programming Languages (POPL)*, Austin, TX, USA, 2011, pp. 317–330.

Millions of spreadsheet users need to reformat, extract, or combine text data. Most lack the programming skills to write the needed formulas or scripts. Gulwani asks whether a system can synthesize these transformations from a few examples. The solution is FlashFill, a programming-by-example (PBE) system that synthesizes string-processing programs from input-output examples. The system defines a domain-specific language (DSL) of string expressions covering concatenation, substring extraction by position or delimiter, and conditionals. It then uses version-space *intersection* to find all DSL programs consistent with the examples and rank them by likelihood. Gulwani evaluates FlashFill on a benchmark of more than 100 string-processing tasks drawn from Excel help-forum posts and the Excel product team. The system typically converges in one to four rounds of interaction, running in less than 0.1 second on average. FlashFill was integrated into Microsoft Excel 2013 and shipped to millions of end users. The system cannot handle tasks that require arithmetic reasoning. It uses only a restricted form of regular expressions (one-or-more occurrences, no disjunction). It also relies on a fixed set of predefined tokens that must be extended manually for new domains. It can also synthesize an unintended program when the examples are ambiguous. Future work could extend the DSL to cover more data types and study how users respond to incorrect synthesized programs.

- [4] M. Hirzel, “Low-code programming models,” *Communications of the ACM*, vol. 66, no. 10, pp. 76–85, Oct. 2023.

Low-code programming spans four areas: visual programming, programming by example, programming by demonstration, and natural-language programming. These areas have largely been

studied separately, with little interaction between them. Hirzel proposes a taxonomy organized around the *notation* used to express programs: visual programming languages, programming by demonstration, and programming by natural language. He surveys systems in each category and identifies shared design challenges such as intent disambiguation and liveness. He also highlights cases where techniques from one category have been transferred to another. The evaluation is qualitative. Hirzel synthesizes findings from the surveyed literature and illustrates the taxonomy with concrete systems such as Scratch, FlashFill, and Codex. No new empirical study is conducted. As a survey, this paper inherits the limitations of its sources and does not contribute new experimental data. The taxonomy also does not yet capture hybrid systems that combine multiple notations, such as a visual interface backed by natural-language refinement. Future work could formalize the design space further and develop metrics for comparing low-code systems across categories.

- [5] T. Lau, S. A. Wolfman, P. Domingos, and D. S. Weld, “Programming by demonstration using version space algebra,” *Machine Learning*, vol. 53, no. 1–2, pp. 111–156, 2003.

Earlier PBD systems like EAGER relied on ad hoc heuristics to guess user intent. Others searched over large program spaces by brute force. Neither approach scaled well. Lau et al. ask how to make PBD systems both efficient and general. The authors propose *version space algebra*, a framework that compactly represents all programs consistent with a user’s demonstrations. The key idea is to decompose the program space into independent subspaces connected by union and join operators. As the user gives more demonstrations, the version space is refined incrementally without recomputing from scratch. The authors implement this framework in SMARTedit, a PBD system for text-editing tasks. They evaluate SMARTedit on a benchmark of 30 repetitive text-editing scenarios. The system correctly learned the intended program from one to three demonstrations in most cases. The main limitation is that the framework requires the program space to decompose into independent subspaces, which does not hold for all domains. SMARTedit is also restricted to text-editing macros and handles conditional branching poorly. Future work could extend the algebra to handle dependent subspaces and richer program constructs like conditionals.

- [6] O. Polozov and S. Gulwani, “FlashMeta: A framework for inductive program synthesis,” in *Proc. ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2015, pp. 107–126.

Building a new PBE system for each domain required designing a new DSL and a new synthesis algorithm from scratch, which was time-consuming and error-prone. Polozov and Gulwani ask whether this process can be generalized into a reusable framework. They propose FlashMeta, a framework for building PBE systems. The developer defines a domain-specific language and provides *witness functions* that describe how to propagate input-output examples through each DSL operator. FlashMeta then automatically derives a synthesis algorithm that searches the DSL using these witness functions, removing the need to write a custom search procedure for each domain. The authors evaluate FlashMeta by re-implementing four existing PBE systems (FlashFill, FlashExtract, FlashNormalize, and FlashRelate) and building one new system, WebExtract, for web data extraction. The re-implementations match or exceed the performance of the hand-tuned originals while requiring significantly less code, with reductions ranging from roughly two to eight times fewer lines. FlashMeta requires each DSL operator to have a decidable witness function. Operators whose behavior cannot be inverted, such as multivariate polynomial roots, cannot be supported. Witness functions must also be written manually for each operator, which

still requires PBE expertise. Future work could automate the generation of witness functions or extend the framework to domains that currently require SMT-based solvers.

- [7] I. E. Sutherland, “Sketchpad: A man-machine graphical communication system,” in *Proc. AFIPS Spring Joint Computer Conference*, 1963, pp. 329–346.

In the early 1960s, all programming was textual and batch-oriented, putting it out of reach for designers, engineers, and other non-programmers. Sutherland asks how humans can communicate procedural and geometric information to a computer without writing code. The solution is Sketchpad, a graphical editor for the TX-2 computer. Users draw geometric shapes on a display using a light pen and define constraints among them. The system maintains a structured representation of the drawing and satisfies constraints like parallelism and equal length through a relaxation algorithm. Users can also define “master” drawings and create “instances,” an idea that anticipates object-oriented programming. Sutherland evaluates the system through case studies. He shows that mechanical linkages, circuit diagrams, and bridge structures can all be created and edited interactively. He also reports a case in which one person spent ten hours building a forty-transistor circuit diagram in Sketchpad. Sutherland concludes that the effort is worthwhile only when the user needs the structured representation itself, not just the visual output. Sketchpad was limited to geometric domains and required specialized hardware (the TX-2 and light pen) that was not widely available. The constraint solver could also slow down for large drawings. Future work would need to generalize the approach beyond geometry and make it run on more accessible hardware.